

Custom YOLO Model Training Guideline

This guide will provide instructions to semi-autonomously create labeled dataset and train a custom YOLO segmentation model. The example will walk through the process of training a YOLOv8s-seg model to detect and segment cubes and cylinders. First, we have to manually label some pictures to train the detection model and then semi autonomously we will train a segmentation model.

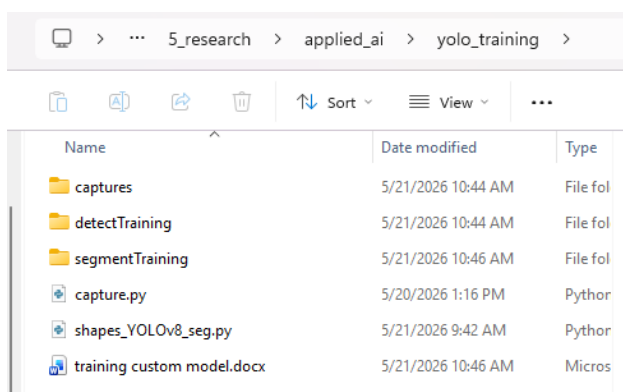
This guide will be using a computer with a NVIDIA Jetson AGX Orin as the training platform. It will also use a windows computer as the platform in which to do the initial image labelling

The full training process is as follows:

Data Collection	2
Custom YOLO Detection Model.....	3
Manual Image Labelling on Windows	3
Getting Files Ready for Training.....	6
Training Model	7
Custom YOLO Segmentation Model.....	8
Auto Image Labelling	8
Manual Verification of Auto Annotated Images	9
Getting Files Ready for Training.....	10
Training Model	10
Testing the Segmentation Model	12

If you just want to test our existing segmentation model using the `shapes_YOLOv8_seg.py` file, you can [download yolov8s-shapes-seg.pt here](#) and go to the last section of this guide.

If you want to follow the example and use the files to test our model or create your own segmentation model, [download all yolo training files from here](#). Add the folders from the zip you downloaded to the `5_research/applied_ai/yolo_training` folder. Your folder should look like the picture below. You can now follow the steps below.



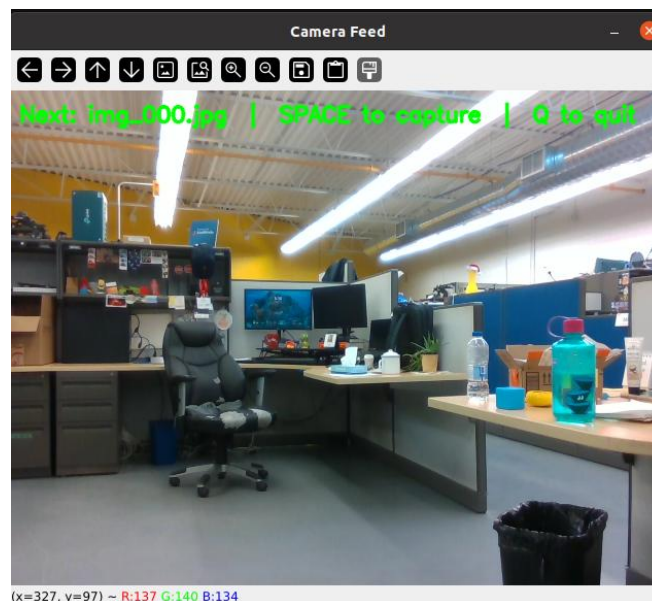
Data Collection

We will train both a detection and a segmentation model. The purpose of training a YOLO detection model first is to then automate the image labelling for the segmentation dataset. The trained detection model will create bounding boxes to crop out the objects of interest (e.g., QCar, Traffic Signs, Cylinders, Cubes or anything else you are interested in finding) such that a Segment Anything Model (SAM) can then be used automatically create the segmentation masks of these objects. The YOLO detection model does not need to be very precise, since the bounding boxes can be bigger than the object of interest. As a result, only a small, labeled dataset is needed.

Although we do not need many pictures of the desired object for the detection model, the segmentation model does. The detection model doesn't need more than 20-40 images, but for the segmentation model, we want closer to 400-500 images. Since you have to take these images, you should take around 400-500 images of your desired objects in a diverse set of environments (background, angles, lighting condition, etc.).

We recommend that the images are taken using the camera of the device you will be using (e.g., QCar or QArm), such that image properties will be consistent between training data and practical usage. However, you can take them either in windows or linux as long as it's the correct camera. Open the `yolo_training` folder in visual studio code. To help taking these pictures, we have provided the file `capture.py` under `yolo_training`.

It will open a window showing you the following output. If the image you are seeing is not from the correct camera you are interested in, click on the Camera Feed window and click Q to close it. Find the `cv2.VideoCapture(#)` line by the top of the script and try different numbers from 0 up and run each time until you find your desired camera. Every time you want to capture an image, press the spacebar while the window is in view. The name of the file shown as `Next:` will update after each image you take. If by any chance you did not take enough pictures or you did not finish taking them, next time you run it, change `START_NUMBER` at the top. That will make sure you don't rewrite files, so if you write 125, your first image taken will be `img_125.jpg`.



Custom YOLO Detection Model

Now that the images are collected, some need to be labelled. Choose around 20-40 pictures total from your dataset of your desired objects in different scenarios and move them into another folder for now. Make sure the files were moved and not copied, so they are not used again when training for segmentation. These images are the ones that will be labelled. We have provided the ones we used for the shapes in the folder `detectTraining/forLabelStudio`.

For reference, the dataset used for training QCar2 detection model contained 16 labeled QCar images for training set and 4 for validation set. The ones used to train cubes and cylinders contained 32 labelled images for training and 5 for validation. The images for the shapes as well as their labels are located in `yolo_training/detectTraining/dataset`.

Manual Image Labelling on Windows

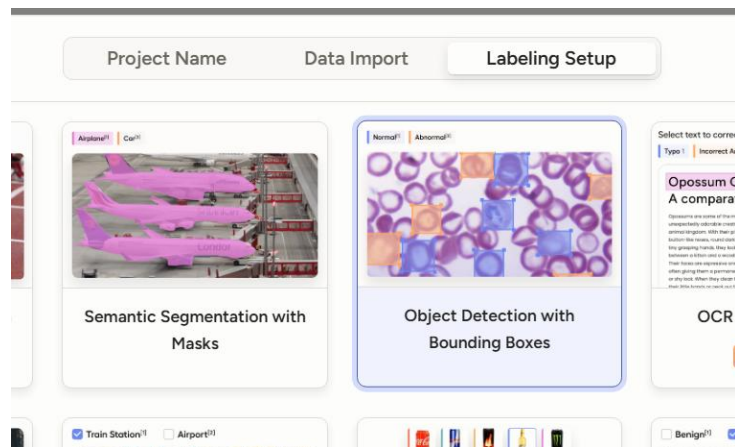
We have tested the following section only in a windows computer. There are many available annotation/labelling tools such as Label Studio and VGG Image Annotator (VIA). We recommend [Label Studio](#) for its ease of use and compatibility with YOLO export formats. To set this up on windows, make sure to create a virtual environment, DO NOT download directly on your python installation without a virtual environment. To download in a virtual environment, follow the steps [here under Install Using Pip](#).

Move the pictures you chose to label (and added to a folder) in the step above to your windows computer. You can use the `forLabelStudio` files we provided for the shapes dataset.

Open label studio by running `label-studio` inside your virtual environment.

You will have to create an account to start this process.

Create a new project, and drag your selected pictures into the Data Import tab. In Labeling Setup, choose Object Detection with Bounding Boxes and click save.

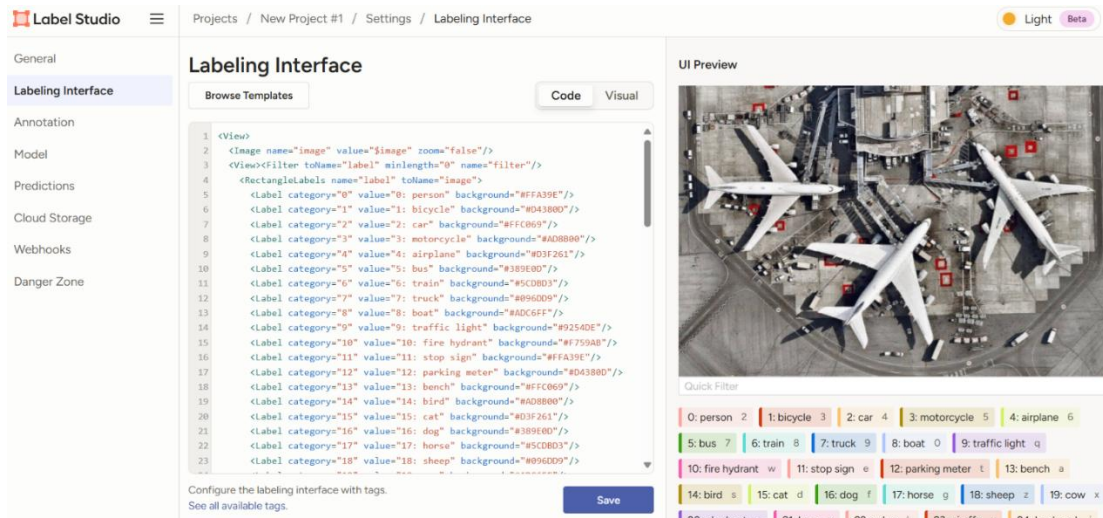


For the labeling Interface, set the window to Code mode and put your desired Labels. We have provided the `"qcar_label_interface.txt"` and `"shapes_label_interface.txt"`, the content of which can be copied to Label Studio to properly configure the classes for your labeling tasks, as shown in the image below.

When doing the QCar training, we followed the COCO dataset and sat the QCar class ID to 2, same class ID of general car in the pretrained model. As such, additional training on the

pretrained model with customized dataset will replace knowledge of general car with QCar. This meant that the model could still recognize all the other parts from the COCO dataset.

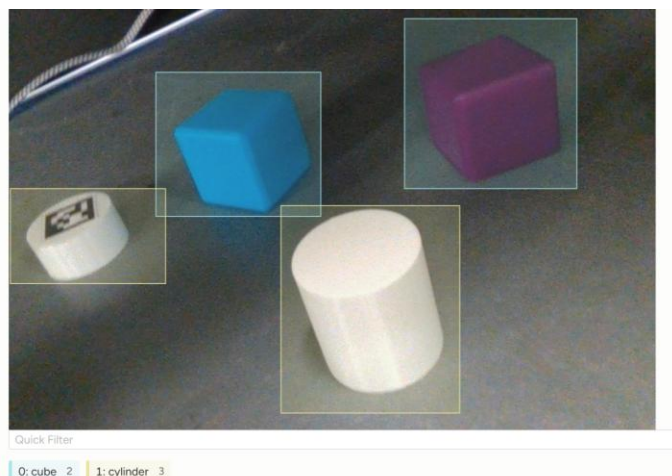
When creating data for cubes and cylinders, we created its own labels and deleted the rest so that the model would only find cubes and cylinders.



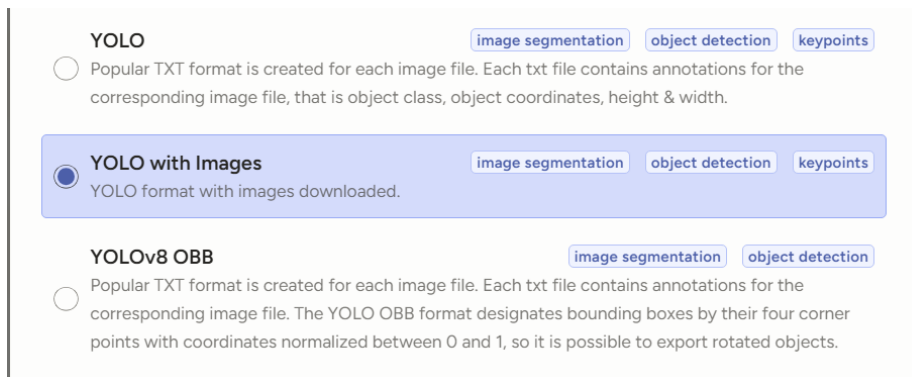
For finding cubes and cylinders, our code looks like this

```
<View>
  <Image name="image" value="$image" zoom="false"/>
  <View><Filter toName="label" minlength="0" name="filter"/>
    <RectangleLabels name="label" toName="image">
      <Label category="0" value="0: cube" background="#a6e6ed"/>
      <Label category="1" value="1: cylinder" background="#ede59f"/>
    </RectangleLabels></View>
</View>
```

You should save your settings, annotate all the images you added by clicking on the labels below each picture and then drawing a bounding box around each desired object as shown on the image below. Note that bounding boxes do not have to be perfect and can have extra space around the object. Make sure you click update once you finish each image.



After annotating all images, go to your project again and export the labels using the "YOLO with Images" option. For each image file, `image.jpg`, a corresponding label file with the same name, `image.txt`, is exported.



YOLO image segmentation object detection keypoints

☐ Popular TXT format is created for each image file. Each txt file contains annotations for the corresponding image file, that is object class, object coordinates, height & width.

YOLO with Images image segmentation object detection keypoints

☒ YOLO format with images downloaded.

YOLOv8 OBB image segmentation object detection

☐ Popular TXT format is created for each image file. Each txt file contains annotations for the corresponding image file. The YOLO OBB format designates bounding boxes by their four corner points with coordinates normalized between 0 and 1, so it is possible to export rotated objects.

Each line in the label file corresponds to one object instance and are in the following format:

```
class_id x_center y_center width height
```

where:

- **class_id**: is an integer representing the object class, starting from 0.
- **x_center**: is the x-coordinate of the bounding box center (normalized between 0 and 1, relative to image width).
- **y_center**: is the y-coordinate of the bounding box center (normalized between 0 and 1, relative to image height).
- **width**: is the width of the bounding box (normalized between 0 and 1, relative to image width).
- **height**: is the height of the bounding box (normalized between 0 and 1, relative to image height).

For example, when using the QCar, an image named `img1.jpg`, should have the corresponding labels saved as `img1.txt`.

```
2 0.503125 0.3171875 0.3104166666666667 0.128125
```



Getting Files Ready for Training

Move the exported folder back to your NVIDIA computer, rename it dataset and place it under `yolo_training/detectTraining/dataset`. The provided files already have the folder in place.

Before training, images and labels should be organized in two separate folders, in which they're further separated into training and validation folders. The export will create a similar folder structure just without the training and validation folders.

Go to your `dataset` folder and create a `train` and a `val` folder. Move around 5 pictures into the `val` folder and the rest into the `train` folder. If you place the `organize_labels.py` file at the same level where your dataset folder is (`yolo_training/detectTraining`), once you run it, it will move the proper files in the labels folder to match the `train` and `val` folder structure so you don't have to manually check the names and make them match.

The proper dataset directory structure once you run that is shown below.

```
dataset/
├── images/
│   ├── train/
│   │   ├── img1.png
│   │   └── img2.png
│   └── val/
│       ├── img3.png
│       └── img4.png
└── labels/
    ├── train/
    │   ├── img1.txt
    │   └── img2.txt
    └── val/
        ├── img3.txt
        └── img4.txt
```

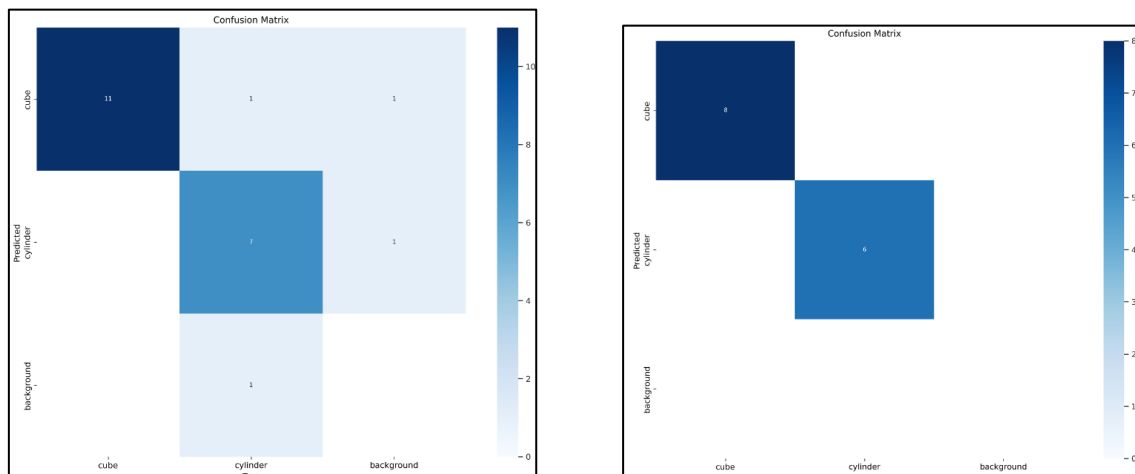
Training Model

The custom model was obtained by continue training a pretrained detection model from Ultralytics on the new dataset. We can utilize the powerful NVIDIA GPU to train the detection model to speed up the process. Make sure all of this is run in your NVIDIA computer and not your windows computer you used to label. Open your `yolo_training/detectTraining` folder.

To begin the training process, a yaml file is needed to configure the training process. We have provided the example `qcar2-bbox.yaml` and the `shapes-bbox.yaml` files which will be used for the detection training using bounding boxes. The QCar2 one includes all the other objects in the COCO dataset and the SHAPES only checks for cubes and cylinders. **We will be using the SHAPES version of the files.** Open `shapes-bbox.yaml` and modify it to include the correct path of the dataset. You can also modify the names of the classes to match your selected ones in the labeling interface.

We have also provided an example training script, `detectTraining/train.py`, for the detection model. Ensure the script includes the correct path the yaml file you will use and run the script. The pre-trained model named "yolov8s.pt" as well as "yolov8n.pt" will be downloaded automatically to your working directory by the script. Note that the training will take around 4-8 minutes.

After running `train.py`, you will have a folder called `runs/train` in your working directory. Look at the latest `train` folder. Look at the `confusion matrix.png` to make sure that your graph shows a diagonal line like the right one, showing that there was no confusion in detecting the objects. If there is a few 1's in other squares that is okay, but you generally want it to be like the picture to the right.



Look at the `train_batch` pictures as well as `val_batch` pictures to see if the objects were found correctly. If not, you could add more pictures to Label studio, annotate the new ones and export the dataset again and repeat all the steps from there again.

Once you are happy with your results, in the `runs/train` folder of your latest run, there will be a weights folder, copy `best.pt` and copy it into your `segmentTraining` folder. Rename it too, since training was done with yolov8s as the model with a bounding box data, we recommend

a name similar to `yolov8s-EXAMPLE-bbox.pt`. In this case, we named it `yolov8s-shapes-bbox.pt`. Copy your `shapes-bbox.yaml` to the `segmentTraining` folder too.

Custom YOLO Segmentation Model

You should have already taken a much more extensive dataset of pictures to train the segmentation model. To train for QCar, we had around 500 pictures and to train for cylinders and blocks we had around 450. **Make sure all of this is run in your NVIDIA computer and not your windows computer you used to label.**

Auto Image Labelling

Move your capture image folder that had all pictures minus the ones you labelled into your `segmentTraining` folder. Open your `segmentTraining` folder in visual studio code. You should have already copied your `shapes-bbox.yaml` and `yolov8s-shapes-bbox.pt` to the `segmentTraining` folder.

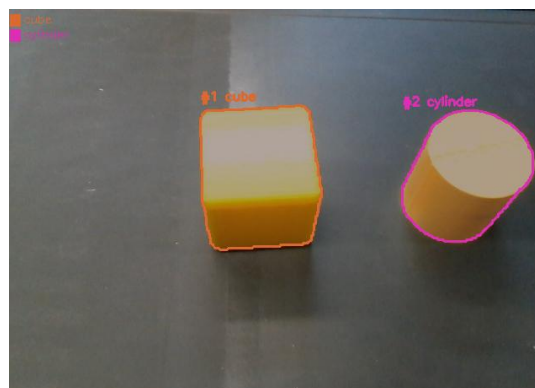
In the automated image labelling process, the detection model will predict the correct class ID and crop out the object, while the SAM (Segment Anything Model) will create the segmentation label of the object. Modify `segmentTraining/auto_annotate.py` to have the correct paths to the images directory (e.g. `"/your_img_dir"`) and YOLO detection model you created in the detection step and run the script. **Note that since provided files are already labelled and moved in the correct format for the next steps, this will not run out of the box.** If you want to run it, copy all files from `dataset/images`, create a folder under `segmentTraining` called `captures` and place them there. Make sure files are not separated into `train` and `val` folders but they are all together.

The SAM model named `"sam_b.pt"` will be downloaded automatically by the script, and the generated segmentation labels will be saved in `"/your_img_dir_auto_annotate_labels"` folder. Similar to the bounding box label files, each line corresponds to one object instance and are in the following format:

```
class_id x1 y1 x2 y2 x3 y3 ... xn yn
```

where (xi, yi) are normalized coordinates of polygon points outlining the object.

```
0 0.5375000238418579 0.2395833283662796 0.535937488079071 0.2416666597127... ...  
1 0.8453124761581421 0.25208333134651184 0.84375 0.2541666626930237 0.8359... ...
```



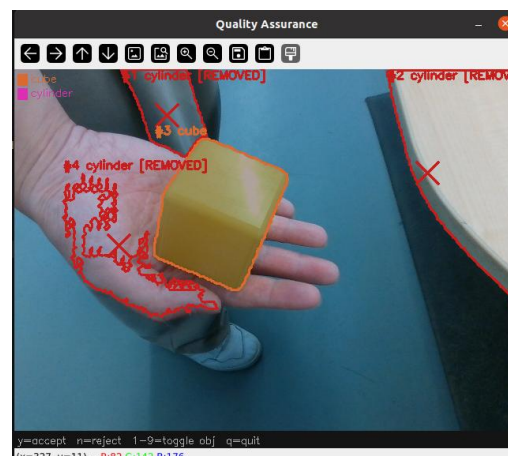
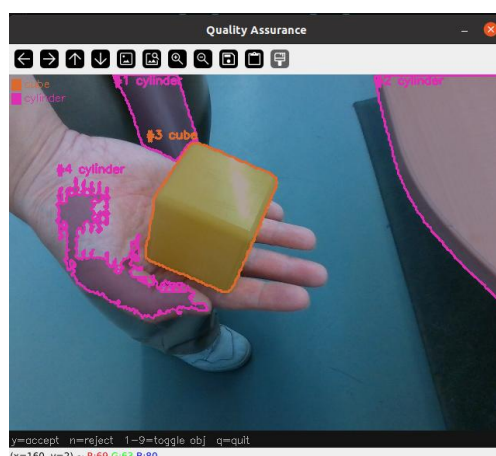
The auto labeling script does not guarantee perfect segmentation labels for the images. Therefore, it is crucial that you go through all the labeled files for quality assurance. Note that

from the original over 490 images, we are only providing around 430 which are the clean images that were labelled properly after manually verifying them.

Manual Verification of Auto Annotated Images

We have provided `segmentTraining/manual_verify.py` that visualizes the segmentation label and allows you to organize the image-label pairs into "good" and "bad" directories. After the file description and the imports at the top there is a Configuration section. Change all 3 values (YAML_PATH, IMAGE_DIR, LABEL_DIR) to point to your correct files, the yaml needs to be the `bbox.yaml` you used for detection training, you should have already copied it to the same level as the manual verify file.

The top of the manual verification file has a description of how to use the file. You will be able to choose the images as a pass or fail by select the pop-up window and pressing "y" key for good labels, and "n" key to move images and labels to the "bad" directories and it will allow you to delete false positives by using the numbers on your keyboard as shown in the pictures below.



If you have a picture where it found the objects correctly, but they were labelled wrong, you could also accept the image, find the corresponding `img_xxx.txt` under the `your_img_dir_auto_annotate_labels` directory and change the first number in the correct line to the correct class ID. You can terminate the script by pressing "q" key, and the verification progress will be saved to `qa_progress_log.txt`. You can resume by running the script again and it will not repeat images you have already done. If you wanted to go through all the images again to confirm the labels, delete the progress log and run the manual verify again, since the bad images already got moved to the bad folder, you should only see the correct ones.

Optionally, you can manually label the "bad" images using Label Studio to enhance the diversity of your dataset.

We are only providing around 430 which are the clean images that were labelled properly after running this script. We are not providing the pictures that went into the `_bad` folder since those were not good pictures or had very bad segmentation.

Getting Files Ready for Training

Copy your `captures` and `captures_auto_annotate_labels` folder, which should now only include the good pictures, into a folder called `dataset`. Rename the folders to `images` and `labels` and in the now called `images` folder, create a `train` and `val` folder. Repeat the steps from the detection model, move 5-10 pictures to the `val` folder for validation. Put the rest in the `train` folder. Copy `organize_labels.py` into the same level where the `dataset` folder is, change the name of the folders and run, this will make sure the `labels` folder is also organized automatically into `train` and `val`.

Your dataset should now follow the same directory structure as the detection training set, as shown below.

```
dataset/
├── images/
│   ├── train/
│   │   ├── img1.png
│   │   └── img2.png
│   └── val/
│       ├── img3.png
│       └── img4.png
└── labels/
    ├── train/
    │   ├── img1.txt
    │   └── img2.txt
    └── val/
        ├── img3.txt
        └── img4.txt
```

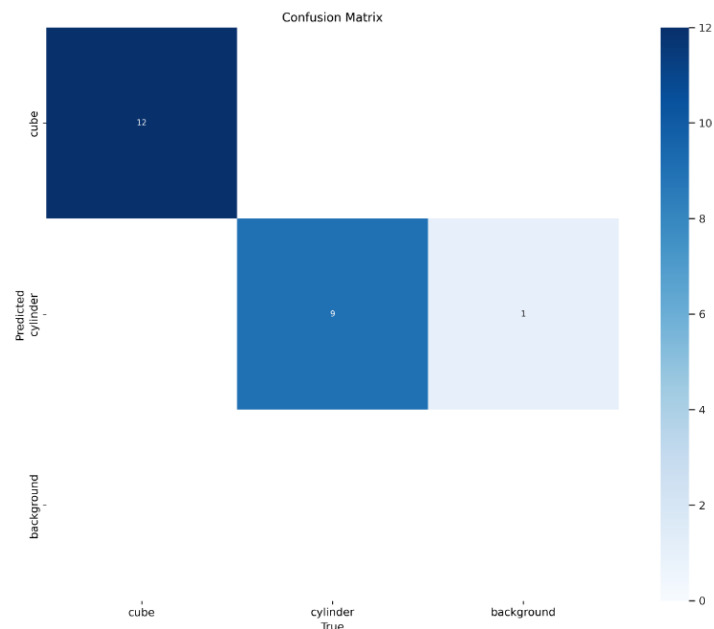
Training Model

To begin the training process for segmentation, a yaml file is needed to configure the training process. We have provided the example `qcar2-seg.yaml` and the `shapes-seg.yaml` files which will be used for the segmentation training. As before, we will be using the SHAPES version of the files. Open `shapes-seg.yaml` and modify it to include the correct paths to your dataset, training and validation images. You can also modify the names of the classes to match your selected ones in the labeling interface.

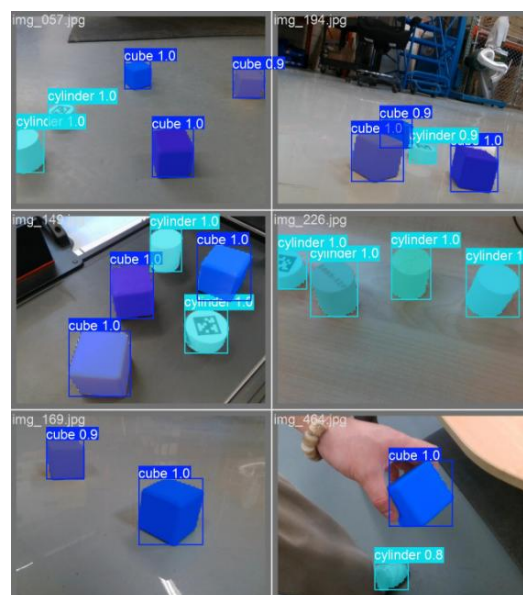
We have also provided an example training script, `segmentTraining/train.py`, for the segmentation model. Ensure the script includes the correct path to the yaml file and run the script. The pre-trained model named "yolov8s-seg.pt" as well as "yolov8n.pt" will be downloaded automatically to your working directory by the script. Note that the training will take around 40-50 minutes.

After running `train.py`, you will have a folder called `runs/segment` in your working directory. Look at the latest `train` folder. Look at the `confusion matrix` to make sure that your graph shows a diagonal line like this, showing that there was no confusion in detecting the objects. If there is 1 or 2 off that is okay but you generally want it to be clean. Find the `best.pt` file

under the `weights` folder, copy it outside of the `segmentTraining` folder inside the `yolo_training` folder and rename it something like `yolov8s-shapes-seg.pt`.



Look at the `train_batch` pictures as well as `val_batch` pictures to see if the objects were found correctly as below. If not, you could add more pictures to the `captures` folder, annotate them again and manually verify them again before training. **If you do this, make sure to add the new pictures in a new folder for the auto annotate to not redo all existing ones**, if not, you will have to do the manual verification of ALL pictures again.



Testing the Segmentation Model

To test the segmentation model that was developed, we have provided the file `yolo_training/shapes_yolov8_seg.py`. This can be run either in windows with a computer with an NVIDIA graphics card or in an NVIDIA computer like the QBrain. If you are running on windows, uncomment the line `# myYolo.net.to('cuda:0')`, it allows the model to run in the NVIDIA GPU instead of on the CPU, making it faster. If running it on Windows, it takes a few seconds to start running, make sure to download Pytorch. For the command to install Pytorch, use the interactive table here <https://pytorch.org/get-started/locally/> to know your compute platform in Windows, run `nvidia-smi` in your terminal and look at the CUDA version.

```
C:\Users\...>nvidia-smi
Thu May 21 09:50:14 2026
```

NVIDIA-SMI 595.71			Driver Version: 595.71			CUDA Version: 13.2		
GPU	Name	Perf	Driver-Model	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute	M.
							MIG	M.
0	NVIDIA RTX 2000 Ada Gene...		WDDM	00000000:01:00.0	On			N/A
N/A	56C	P8	6W / 63W	1717MiB / 8188MiB		2%	Default	N/A

If running in an NVIDIA computer (running Linux), the first time running the file, it will convert the `yolov8s-shapes-seg.pt` into a tensor RT file, creating a `.engine` and `.onnx` file so it can run faster, these files will be located next to the `seg.pt` file. This process takes around 15 minutes for the shapes model, but every subsequent run will not need that time at the beginning. Subsequent runs might just need a just a few seconds before it starts and they will use the `.engine` file by default.

The file provided uses the function `post_processing_shapes()` which outputs a fixed (5,7) array. If fewer than 5 objects are detected, the extra rows become 0. The columns are [class id, centroid X, centroid Y, Hue (median), Saturation (median), Value (median), Blob pixel size]. The rows are sorted from largest to smallest object.

Note, if running the QCar model to find QCars and street signs, the example is under `5_research\sdc\qcar2\hardware\applications\yolo`.